BOSS-KAMPF 4 · Welt 4 · DS 30 · 60 min · Hilfsmittel: Werkzeugkarten + ADT-Schnittstellen (EH §4)

W4·Boss – Die Fähre von Nebelhafen

Erreichbar: 24 BE · Papier-Compiler zu c) im Anschluss · Zusätzlich im Lab: **Online-Boss „Die Postkutschenstation“**
Niveau (KC 2017): Grund- und Leistungskurs (gA und eA)

Im Nebel vor der Hafenstadt pendelt die Fähre **Nebelstern** zur Insel. Fahrzeuge warten am Anleger, fahren über eine einzige Heckrampe auf das Deck und stehen vorher in der Buchungsliste der Reederei. Die Verwaltung übernimmt die Klasse **Faehre** (Abbildung 1) – mit einem Stapel, einer Schlange und einer dynamischen Reihung. Zeig, dass du die drei Strukturen sicher und ohne Laufzeitfehler einsetzen kannst!

Hinweise zur Bearbeitung

- Bearbeite alle Teilaufgaben **ohne Gerät** auf dem Antwortbogen am Ende dieses Hefts. Erlaubt sind deine Werkzeugkarten und die ADT-Schnittstellen im Material.
- Implementierungen schreibst du in **Java** mit den Klassen Stack, Queue und DynArray der Bibliothek „Niedersachsen“ – ohne Generics, Rückgaben werden **gecastet**, z. B. `(Fahrzeug) deck.pop()`. Struktogramme nach den Abi-Konventionen.
- Laufzeitfehler musst du durch **explizite Abfragen** ausschließen: vor pop/top/dequeue/head „Leer?“, vor getItem/setItem/delete „Index?“.
- Zeitbudget wie im Abitur: ca. 2,5 Minuten pro BE. Die BE stehen rechts neben jeder Teilaufgabe.

Papier-Compiler und Online-Boss

Deine Lösung zu **Aufgabe c)** tippst du direkt nach dem Boss-Kampf **unverändert** in der Online-IDE ab und startest die Tests im Testrunner. Kreuze im Antwortbogen an, ob alle Tests beim ersten Versuch oder erst nach einer Korrektur grün waren. Typische Fallen: fehlende Leer-Abfrage vor pop, Deck am Ende nicht unverändert.

Zusätzlich im Lab: Online-Boss „Die Postkutschenstation“ (25 min, Auswertung am Ende, der beste Versuch zählt) – Punkte gibt es dort automatisch.

Aufgaben

a) Zum Testen arbeitet die Hafenmeisterin mit einem Stapel deck und einer Schlange anleger, beide vom Inhaltstyp Zeichenkette (Abbildung 3).

Stellen Sie im Antwortbogen den Inhalt von deck und anleger zu den Zeitpunkten 1, 2 und 3 dar und geben Sie die Ausgabe von Zeile 16 an.

Erläutern Sie, warum in Zeile 13 die Abfrage `!anleger.isEmpty()` notwendig ist und warum sie vor dem Aufruf von head() stehen muss. **[4 BE]**

b) Die Reederei lässt eine zweite Fähre bauen. Auf sie fahren die Fahrzeuge über eine Heckrampe auf und verlassen sie über eine Bugrampe.

Begründen Sie, welche der Datenstrukturen Stapel, Schlange und dynamische Reihung das Deck dieser Fähre am besten modelliert.

Ein Mitarbeiter schlägt vor, auch die Buchungsliste als Schlange zu verwalten, weil die Fahrzeuge ohnehin der Reihe nach verladen werden. Neue Buchungen werden jedoch nach Länge einsortiert (Aufgabenteil d), und bei Stornierungen werden Fahrzeuge aus der Mitte der Liste entfernt. Beurteilen Sie diesen Vorschlag. **[3 BE]**

c) Die Fähre **Nebelstern** hat nur eine Heckrampe: Bevor ein Fahrzeug das Deck verlassen kann, müssen alle Fahrzeuge abfahren, die nach ihm aufgefahren sind. Die Operation **ausfahrtBis** liefert die Anzahl der Fahrzeuge, die vor dem Fahrzeug mit dem übergebenen Kennzeichen das Deck verlassen müssen. Befindet sich kein Fahrzeug mit diesem Kennzeichen auf dem Deck, wird −1 geliefert. Nach Ausführung der Operation muss das Deck unverändert sein (dieselben Fahrzeuge in derselben Reihenfolge).

Beispiel: Für die Fähre aus Abbildung 2 liefert `ausfahrtBis("OL-B 9")` den Wert 3 und `ausfahrtBis("NH-K 7")` den Wert 0.

Implementieren Sie die Operation ausfahrtBis in Java. Laufzeitfehler sind durch geeignete Abfragen auszuschließen.

[6 BE]

d) Die Buchungsliste ist absteigend nach der Länge der Fahrzeuge sortiert, weil lange Fahrzeuge zuerst verladen werden (Abbildung 2). Die Operation **einbuchen** fügt das Fahrzeug f so in die Buchungsliste ein, dass sie sortiert bleibt; f wird hinter allen Fahrzeugen derselben Länge eingefügt.

Beispiel: Ein Fahrzeug „NH-N 9“ mit 7 m Länge steht nach dem Einbuchen in der Buchungsliste aus Abbildung 2 an Index 4.

Entwickeln Sie ein Struktogramm für die Operation einbuchen. Laufzeitfehler sind auszuschließen.

[4 BE]

e) Vor dem Verladen werden die Fahrzeuge am Anleger mit der Operation **umsortieren** (Abbildung 4) neu angeordnet.

Stellen Sie die Verarbeitung des Aufrufs `umsortieren(10)` für die Fähre aus Abbildung 2 dar, indem Sie die Tracetabelle im Antwortbogen vervollständigen.

Erläutern Sie die Funktionsweise der Operation umsortieren. Beschreiben Sie, wie die Operation verändert werden muss, damit auch die langen Fahrzeuge ihre ursprüngliche Reihenfolge behalten.

[4 BE]

f) Die Methode **verladen** (Abbildung 5) soll alle Fahrzeuge der Buchungsliste mit einer Länge von höchstens maxLaenge in der Reihenfolge der Liste auf das Deck fahren lassen, sie aus der Buchungsliste entfernen und ihre Anzahl liefern. Beim Aufruf `verladen(7)` für die Fähre aus Abbildung 2 kommt es zu einem Laufzeitfehler.

Analysieren Sie den Ablauf dieses Aufrufs. Erläutern Sie die Ursache des Fehlers und seine Auswirkungen und korrigieren Sie die Methode.

[3 BE]

Summe: 24 BE

Material

zu den Aufgabenteilen c) bis f)

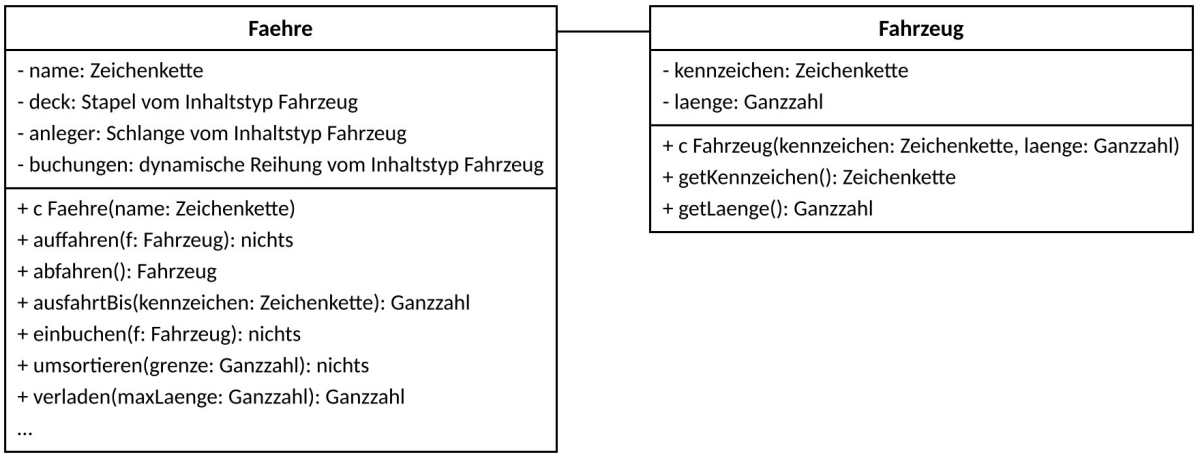
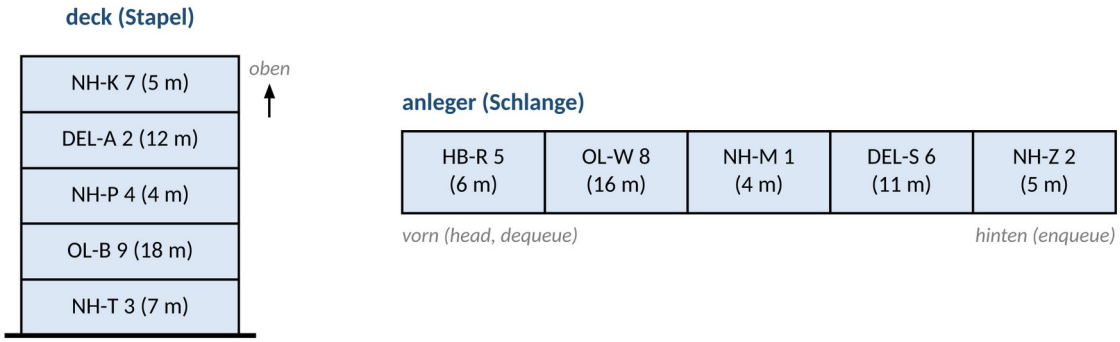


Abbildung 1: Klassenkarten der Klassen Faehre und Fahrzeug (laenge in Metern)



Index	0	1	2	3	4
buchungen	HB-F 4 (17 m)	OL-C 3 (12 m)	NH-E 8 (7 m)	DEL-G 1 (7 m)	NH-H 6 (4 m)

Abbildung 2: Zustand der Fähre nebelstern: deck als Turm (oben steht das zuletzt aufgefahrene Fahrzeug), anleger als Reihe (vorn steht das zuerst angekommene Fahrzeug) und die Buchungsliste buchungen. Angegeben sind Kennzeichen und Länge.

zu Aufgabenteil a)

```
1   Stack deck = new Stack();
2   Queue anleger = new Queue();
3   anleger.enqueue("Bus");
4   anleger.enqueue("Lkw");
5   anleger.enqueue("Pkw");
6   deck.push(anleger.dequeue());
7   deck.push(anleger.dequeue());
    // ---------------------------------- Zeitpunkt 1
8   anleger.enqueue("Rad");
9   anleger.enqueue(deck.pop());
10  deck.push(anleger.dequeue());
11  anleger.enqueue(anleger.dequeue());
    // ---------------------------------- Zeitpunkt 2
12  String x = (String) deck.top();
13  while (!anleger.isEmpty() && !((String) anleger.head()).equals(x)) {
14      deck.push(anleger.dequeue());
15  }
16  println(x + " " + deck.top());
    // ---------------------------------- Zeitpunkt 3
```

Abbildung 3: Befehlsfolge mit einem Stapel und einer Schlange (Online-IDE, Bibliothek „Niedersachsen“)

zu Aufgabenteil e)

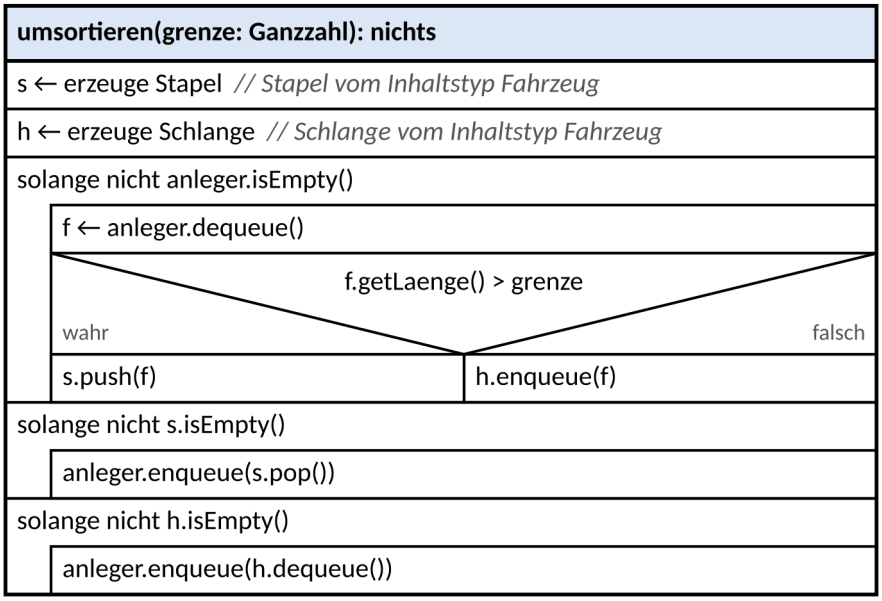


Abbildung 4: Struktogramm der Operation umsortieren der Klasse Faehre

zu Aufgabenteil f)

```
1   public int verladen(int maxLaenge) {
2       int anzahl = 0;
3       int n = buchungen.getLength();
4       for (int i = 0; i < n; i++) {
5           Fahrzeug f = (Fahrzeug) buchungen.getItem(i);
6           if (f.getLaenge() <= maxLaenge) {
7               deck.push(f);
8               buchungen.delete(i);
9               anzahl++;
10          }
11      }
12      return anzahl;
13  }
```

Abbildung 5: Implementierung der Methode verladen der Klasse Faehre

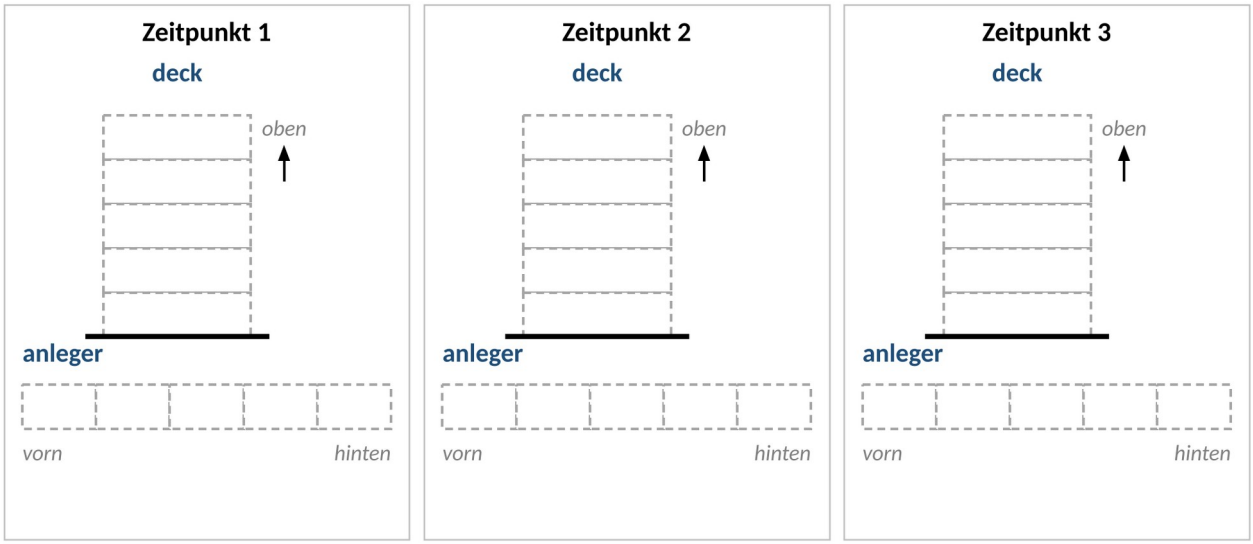
Hilfsmittel: ADT-Schnittstellen (Auszug nach EH §4)

Datenstruktur	Operationen	Beschreibung
Stapel (Stack)	c Stack() · isEmpty(): Wahrheitswert push(x: Inhaltstyp): nichts pop(): Inhaltstyp top(): Inhaltstyp	push legt x oben auf den Stapel. pop entfernt das oberste Element und liefert es, top liefert es, ohne es zu entfernen – beide nur bei nicht leerem Stapel.
Schlange (Queue)	c Queue() · isEmpty(): Wahrheitswert enqueue(x: Inhaltstyp): nichts dequeue(): Inhaltstyp head(): Inhaltstyp	enqueue stellt x hinten an. dequeue entfernt das vorderste Element und liefert es, head liefert es, ohne es zu entfernen – beide nur bei nicht leerer Schlange.
dynamische Reihung (DynArray)	c DynArray() · isEmpty(): Wahrheitswert getLength(): Ganzzahl getItem(i: Ganzzahl): Inhaltstyp append(x: Inhaltstyp): nichts insertAt(i: Ganzzahl, x: Inhaltstyp): nichts setItem(i: Ganzzahl, x: Inhaltstyp): nichts delete(i: Ganzzahl): nichts	Gültige Indizes: 0 bis getLength() − 1. append hängt hinten an. insertAt fügt an Position i ein, die folgenden Elemente rücken nach hinten (i = getLength() hängt an). setItem ersetzt das Element an Position i. delete entfernt es, die folgenden Elemente rücken nach vorn.

In der Online-IDE (Bibliothek „Niedersachsen“) heißen die Klassen Stack, Queue und DynArray. Sie sind nicht generisch: Inhalte haben den Typ Object und werden beim Entnehmen gecastet, z. B. `Fahrzeug f = (Fahrzeug) anleger.dequeue();` oder `String s = (String) deck.top();`.

Antwortbogen

a) Zustände von deck und anleger (Kästen von unten bzw. von vorn füllen) · Ausgabe · Erläuterung **[4 BE]**



Ausgabe von Zeile 16: _____

b) Deck der neuen Fähre · Beurteilung des Vorschlags **[3 BE]**

c) Implementierung von ausfahrtBis – Papier-Compiler: sauber und lesbar schreiben! **[6 BE]**

d) Struktogramm der Operation einbuchen **[4 BE]**

Zeichenfläche für das Struktogramm (Kopf mit Signatur nicht vergessen)

e) Tracetabelle zu umsortieren(10) – Kennzeichen genügen, leere Struktur: – **[4 BE]**

f (Kennzeichen)	anleger (vorn zuerst)	s (oben zuerst)	h (vorn zuerst)
vor der 1. Schleife	HB-R 5, OL-W 8, NH-M 1, DEL-S 6, NH-Z 2	–	–
nach der 2. Schleife			
nach der 3. Schleife			

Funktionsweise · Veränderung für die langen Fahrzeuge:

f) Ablauf von verladen(7) · Ursache und Auswirkungen · Korrektur **[3 BE]**

Auswertung (Peer-Korrektur nach Kernpunkten)

Teilaufgabe	a)	b)	c)	d)	e)	f)	Summe
max. BE	4	3	6	4	4	3	24
erreicht							

Korrigiert von: _____ Papier-Compiler c): ☐ grün beim 1. Versuch ☐ grün nach Korrektur ☐ nicht grün